

## SIMATS ENGINEERING

### CO1 - ASSESSMENT TOOL 3

#### EXPERIMENTS

|                            |                                |
|----------------------------|--------------------------------|
| <b>Name of the Student</b> | <b>S. Yuva Guru Mani Varma</b> |
| <b>Reg. No</b>             | <b>192425005</b>               |

**COURSE NAME:** Deep Learning  
**FACULTY :** Dr.Arul Raj A.M

**COURSE CODE:** MLA0403

---

#### COURSE OUTCOME COVERED

**CO 1:** Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

**CO1 Weightage:** 15%

**Assessment Tool Weightage:** 35%

**Duration:** 30 minutes

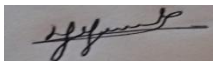
**Marks:** 50

---

#### Code of Conduct:

I S. Yuva Guru Mani Varma/192425005 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



**Signature of the student:**

**Name of the student:** S. Yuva Guru Mani Varma

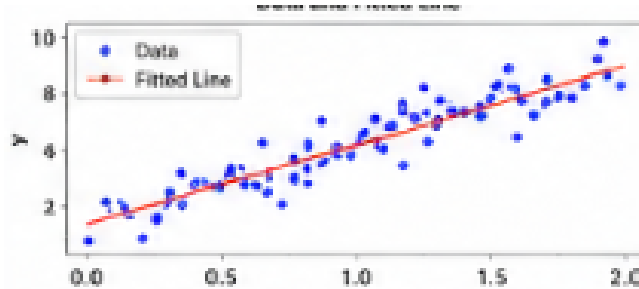
**CO1- AT3 Experiments**  
**Questions**

## 1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

**Answer :**

Generate a dataset with input  $X$  and output  $Y$ .



- Initialize weights randomly.
- Update weights using Gradient Descent.
- Compute Mean Squared Error (MSE) after each iteration.
- Plot Loss vs Iterations.
- Observation: Loss decreases gradually and converges to a minimum, indicating successful learning.

## 2. Maximum Likelihood Estimation (MLE)

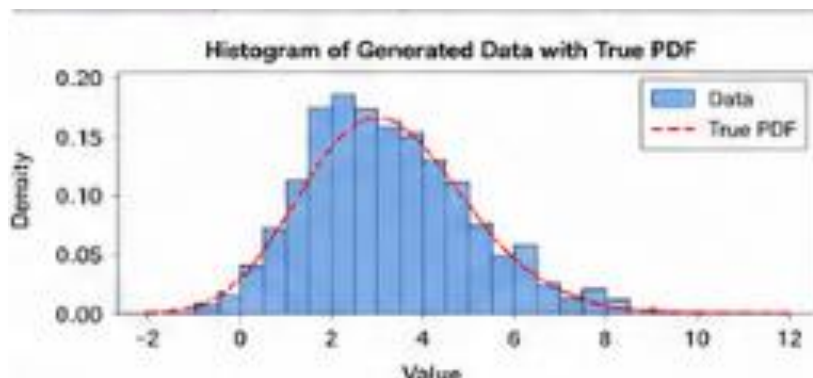
Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

**Answer :**

- Generate synthetic data from a normal distribution  $N(\mu, \sigma^2)$ .
- Estimate

$$\hat{\mu} = \frac{1}{n} \sum x_i$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \hat{\mu})^2$$



- Compare estimated values with actual parameters.

- Observation: Estimated mean and variance are very close to the true values as sample size increases.

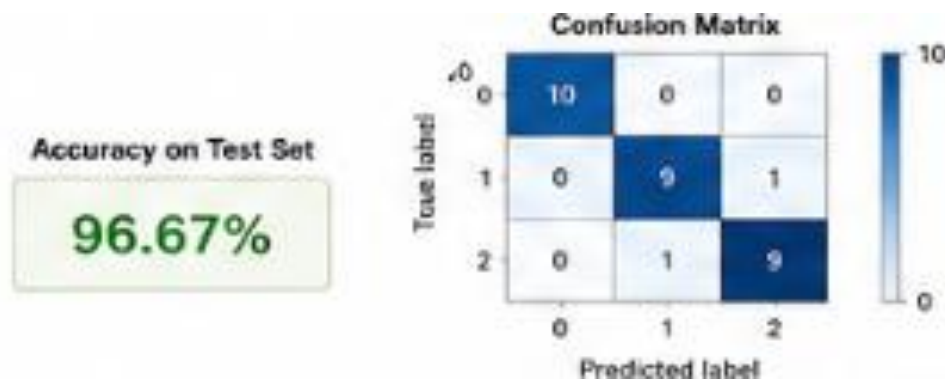
### 3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

**Answer :**

Steps:

1. Load dataset (e.g., Iris).
2. Handle missing values.
3. Normalize features.
4. Split into train and test sets.
5. Train a classifier (Logistic Regression/Decision Tree).
6. Predict test samples.
7. Evaluate using:
  - Accuracy
  - Confusion Matrix



Observation:

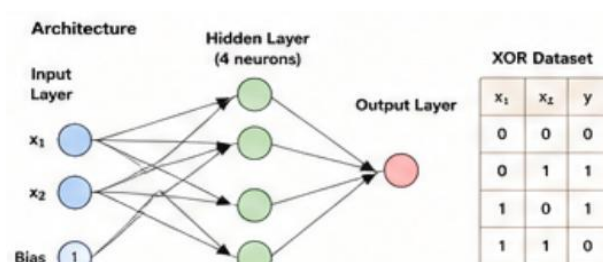
High accuracy and diagonal confusion matrix indicate good classification performance.

### 4. Neural Networks

**Q4.** Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

**Answer :**

1. Build a neural network with:
2. Input layer
3. One hidden layer
4. Output layer
5. Train on XOR dataset.
6. The hidden layer enables learning of non-linear decision boundaries.



- Loss decreases over epochs, showing successful learning.

## 5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

**Answer :**

Implement a single neuron using:

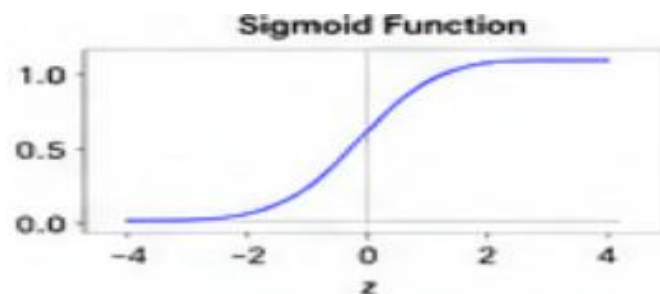
Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}}$$

ReLU

$$f(x) = \max(0, x)$$

Comparison



**Sigmoid**

Output between 0 and 1

Smooth curve

Suffers vanishing gradient

**ReLU**

Output  $\geq 0$

Fast computation

Preferred in deep learning

## 6. Perceptron and Multilayer Perceptron (MLP)

- Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

**Answer :**

(a) Perceptron

Answer:

- Train on a linearly separable dataset.
- Perceptron converges successfully.
- On non-linearly separable datasets (e.g., XOR), it fails to converge.

(b) MLP

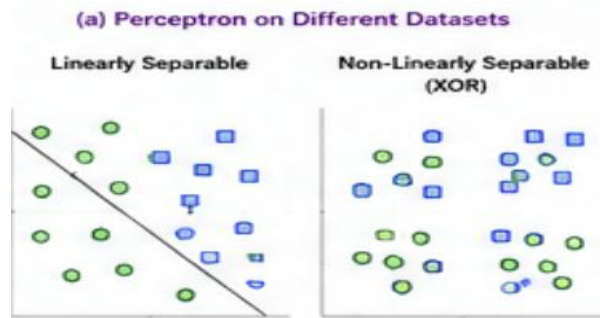
Answer:

- Train an MLP with one hidden layer.
- Compare with Perceptron.

Observation

- Perceptron works only for linear problems.

- MLP successfully learns non-linear patterns and achieves higher accuracy.



- b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

**Answer :**

(b) MLP

Answer:

- Train an MLP with one hidden layer.
  - Compare with Perceptron.
- Observation
- Perceptron works only for linear problems.
  - MLP successfully learns non-linear patterns and achieves higher accuracy.

## 7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.
- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

**Answer :**

### (a) Forward Propagation

- Given inputs, weights and bias:
  - Compute hidden layer outputs.
  - Apply activation function.
  - Compute final output.
- Verify calculations using Python.

### (b) Backpropagation

**Answer:**

- Compute output error.
- Calculate gradients.
- Update weights using:

## 8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

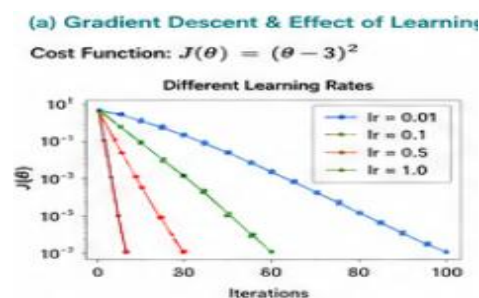
**Answer :**

### (a) Gradient Descent

- Minimize the cost function.
- Compare learning rates.

**Observation:**

- Small learning rate  $\rightarrow$  slow convergence.
- Very large learning rate  $\rightarrow$  divergence.
- Moderate learning rate  $\rightarrow$  fastest stable convergence.



b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

**Answer :**

### (b) Stochastic Gradient Descent

- Updates parameters after every training sample.
- Converges faster than Batch Gradient Descent.
- Loss fluctuates due to noisy updates.

## 9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

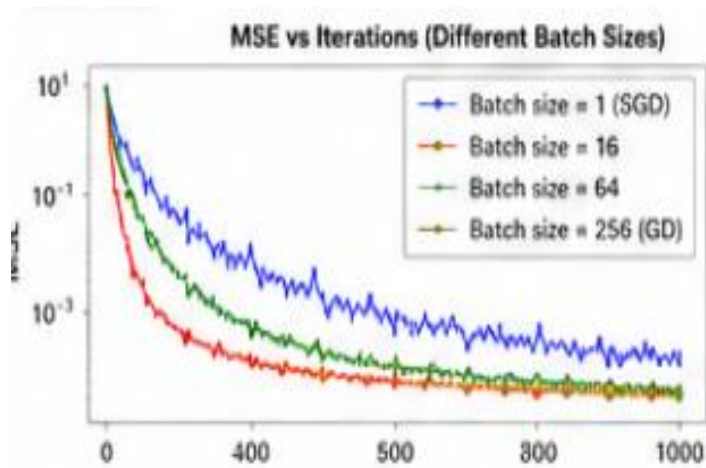
**Answer :**

Train with different batch sizes.

Observation

- Batch size = 1 (SGD): Fast but noisy.
- Batch size = 16–64: Best balance.
- Batch size = Full dataset: Stable but slow.

Mini-Batch GD is generally preferred because it balances speed and stability.



## 10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

**Answer :**

Create datasets with increasing dimensions.

Analyze:

- Average distance between points.
- Classification accuracy.

Observation

- Distances become increasingly similar.
- Distance-based algorithms become less effective.
- Model accuracy decreases due to sparse data and overfitting.

Solutions:

- PCA
- Feature Selection
- L1/L2 Regularization
- Autoencoders

